

Jenkins CI/CD Lab — Practice Answers (Set 2)

ISWE406L — Agile Development Process and DevOps | Windows Agent

Question 1 — Checkout / Install Dependencies / Run Unit Tests

app.py

```
def square(n):  
    return n * n  
  
def is_even(n):  
    return n % 2 == 0
```

test_app.py

```
from app import square, is_even  
  
def test_square_positive():  
    assert square(4) == 16  
  
def test_square_negative():  
    assert square(-3) == 9  
  
def test_is_even_true():  
    assert is_even(8) is True  
  
def test_is_even_false():  
    assert is_even(7) is False
```

requirements.txt

pytest

Jenkinsfile

```
pipeline {  
    agent { label 'windows' }  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'  
            }  
        }  
        stage('Install Dependencies') {  
            steps {  
                bat 'pip install -r requirements.txt'  
            }  
        }  
    }  
}
```

```

    stage('Run Unit Tests') {
        steps {
            bat 'pytest test_app.py'
        }
    }
}

```

First run (correct code). The Console Output ends with a summary line such as:

4 passed in 0.04s

confirming all four test functions (two for square, two for is_even) passed.

After breaking is_even. Change it to something wrong, e.g.:

```

def is_even(n):
    return n % 2 == 1    # bug: logic inverted

```

Push the change and rebuild. pytest now reports 2 passed, 2 failed (both is_even tests fail while both square tests still pass). Because pytest exits with a non-zero status, the **Run Unit Tests** stage fails — Jenkins marks that stage red in the stage view and the pipeline stops there; it never proceeds past it (there is no later stage in this pipeline, so the build itself finishes as **FAILURE** at the Run Unit Tests stage).

Question 2 — Parametrized Tests (Verbose)

app.py

```

def sum_list(numbers):
    return sum(numbers)

def count_negatives(numbers):
    return sum(1 for n in numbers if n < 0)

```

test_app.py

```

import pytest
from app import sum_list, count_negatives

@pytest.mark.parametrize("numbers, expected", [
    ([1, 2, 3], 6),
    ([-1, -2, -3], -6),
    ([0, 0, 0], 0),
    ([10, -5, 5], 10),
])
def test_sum_list(numbers, expected):
    assert sum_list(numbers) == expected

@pytest.mark.parametrize("numbers, expected", [
    ([1, -2, 3], 1),
    ([-1, -2, -3], 3),
    ([1, 2, 3], 0),
])
def test_count_negatives(numbers, expected):
    assert count_negatives(numbers) == expected

```

Jenkinsfile

```
pipeline {
  agent { label 'windows' }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
      }
    }
    stage('Install Dependencies') {
      steps {
        bat 'pip install -r requirements.txt'
      }
    }
    stage('Run Unit Tests (verbose)') {
      steps {
        bat 'pytest -v test_app.py'
      }
    }
  }
}
```

Console Output. `sum_list` has 4 parametrized cases and `count_negatives` has 3, so `pytest` collects and reports 7 passed in total. With `-v`, each case is labelled with the function name plus an auto-generated index/value suffix in square brackets, e.g.:

```
test_app.py::test_sum_list[numbers0-6] PASSED
test_app.py::test_sum_list[numbers1--6] PASSED
test_app.py::test_sum_list[numbers2-0] PASSED
test_app.py::test_sum_list[numbers3-10] PASSED
test_app.py::test_count_negatives[numbers0-1] PASSED
test_app.py::test_count_negatives[numbers1-3] PASSED
test_app.py::test_count_negatives[numbers2-0] PASSED
```

```
===== 7 passed in 0.05s =====
```

Adding a wrong case. Add e.g. `([5, 5, 5], 99)` to `test_count_negatives`. Rerunning shows 7 passed, 1 failed: every `test_sum_list` case still passes, and the first three `test_count_negatives` cases still pass, but the new fourth case (`test_count_negatives[numbers3-99]`) fails with an assertion error showing the real count (0) versus the wrong expected value (99) — confirming each parametrized row is checked independently.

Question 3 — Choice Parameter + Deploy Approval Gate

app.py

```
def main():
    print("Deploying application")

if __name__ == "__main__":
    main()
```

Jenkinsfile

```

pipeline {
  agent { label 'windows' }

  parameters {
    choice(name: 'ENVIRONMENT', choices: ['dev', 'staging', 'prod'], description: 'Target environment')
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
      }
    }
    stage('Build') {
      steps {
        bat 'python -m py_compile app.py'
      }
    }
    stage('Deploy') {
      steps {
        input message: "Approve deployment to ${params.ENVIRONMENT}?", ok: 'Go'
        bat 'python app.py'
      }
    }
  }
}

```

Build 1 — staging, click Go. The build starts with ENVIRONMENT=staging chosen from **Build with Parameters**. It compiles cleanly, then pauses on “*Approve deployment to staging?*”. Clicking **Go** resumes the pipeline, python app.py runs and prints Deploying application, and the build ends **SUCCESS**.

Build 2 — prod, click Abort. The build starts with ENVIRONMENT=prod. It again compiles and pauses on “*Approve deployment to prod?*”. Clicking **Abort** interrupts the flow before python app.py ever runs; the Deploy stage and the build are marked **ABORTED**.

Parameter choice vs. input approval. Choosing ENVIRONMENT happens **before** the pipeline starts — it is a static value read once at build-trigger time and substituted wherever params.ENVIRONMENT is referenced; it cannot change mid-run. The input step, by contrast, is a **runtime pause**: the pipeline has already begun executing, real work (the Build stage) has already happened, and a human decision made *during* that specific run determines whether the remaining steps execute at all. In short, a parameter configures what the run will do; an input step gates whether the run is allowed to continue doing it.

Question 4 — Build Metadata + Per-Build Artifacts

app.py

```

COURSE_NAME = "ISWE406L"
STUDENTS_ENROLLED = 42 # change this value between builds

def main():
    with open("build_report.txt", "w") as f:
        f.write(f"Course: {COURSE_NAME}\n")

```

```

        f.write(f"Students enrolled: {STUDENTS_ENROLLED}\n")
    print("Report generated")

if __name__ == "__main__":
    main()

```

Jenkinsfile

```

pipeline {
    agent { label 'windows' }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Generate Report') {
            steps {
                echo "BUILD_NUMBER: ${env.BUILD_NUMBER}"
                echo "JOB_NAME: ${env.JOB_NAME}"
                echo "WORKSPACE: ${env.WORKSPACE}"
                bat 'python app.py'
            }
        }
        stage('Archive Report') {
            steps {
                archiveArtifacts artifacts: 'build_report.txt', fingerprint: true
            }
        }
    }
}

```

Two builds, changed student count. For build #1, STUDENTS_ENROLLED = 42 produces a build_report.txt archived under build #1. Change the value (e.g. to 50), push, and trigger build #2 — its own build_report.txt is generated and archived separately under build #2. Opening build #1's page afterward still shows its archived report reading 42, while build #2's page shows 50: each build's artifact is stored against that build number rather than being overwritten, since archiveArtifacts (with fingerprint: true) records a distinct, retrievable copy per build.

Question 5 — Parallel Checks + Post Conditions

unit_check.py

```

import time

print("Unit check started")
time.sleep(5)
print("Unit check passed")

```

integration_check.py

```

import time

```

```
print("Integration check started")
time.sleep(5)
print("Integration check passed")
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Parallel Checks') {
            parallel {
                stage('Unit Check') {
                    steps {
                        bat 'python unit_check.py'
                    }
                }
                stage('Integration Check') {
                    steps {
                        bat 'python integration_check.py'
                    }
                }
            }
        }
        stage('Summary') {
            steps {
                echo 'All checks completed.'
            }
        }
    }

    post {
        success {
            echo 'PIPELINE SUCCESS: all checks passed.'
        }
        failure {
            echo 'PIPELINE FAILED: one or more checks did not pass.'
        }
    }
}
```

Timing estimate. Each script sleeps ~5 seconds. Sequentially, the two would take roughly **10 seconds** combined. Running them in parallel {}, both start at once on separate executors, so the Parallel Checks stage takes roughly **5 seconds** — bounded by the slower branch rather than the sum of both.

After a syntax error in integration_check.py. Rebuilding now causes python integration_check.py to exit with a non-zero status due to the syntax error. Because one branch of a parallel block failing fails the whole Parallel Checks stage, Jenkins does **not** proceed to the Summary stage — it is skipped, since by default a failed stage stops the remaining pipeline stages. The post block then evaluates its failure condition (not success), printing:

PIPELINE FAILED: one or more checks did not pass.

Question 6 — Environment Block + Conditional Notification

app.py

```
def main():
    print("Build script running")

if __name__ == "__main__":
    main()
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    environment {
        APP_NAME      = 'MyApp'
        APP_VERSION   = '2.1.0'
    }

    parameters {
        booleanParam(name: 'SEND_EMAIL', defaultValue: false, description: 'Send a notification')
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                echo "Built ${env.APP_NAME} version ${env.APP_VERSION}"
            }
        }
        stage('Send Notification') {
            when {
                expression { return params.SEND_EMAIL }
            }
            steps {
                script {
                    def subject = "${env.APP_NAME} ${env.APP_VERSION} - Build Notification"
                    try {
                        mail to: 'you@example.com', subject: subject, body: "Build URL: ${env.BUILD_URL}"
                    } catch (err) {
                        echo "SMTP not configured – Subject: ${subject}"
                    }
                }
            }
        }
    }
}
```

```
}
```

SEND_EMAIL checked. All three stages run; Send Notification executes and either sends the mail or prints the echo fallback containing MyApp 2.1.0 in the subject.

SEND_EMAIL unchecked. Checkout and Build run normally, but the when condition on Send Notification evaluates to false, so that stage is **skipped** — Jenkins shows it in the stage view marked as skipped (grey), and the overall build still finishes as **SUCCESS**.

Skipped vs. failed. A **skipped** stage is one Jenkins deliberately did not attempt, because a when condition decided it wasn't applicable to this run — it has no pass/fail outcome of its own and does not affect the overall build result. A **failed** stage is one that was attempted but returned a non-zero exit status or threw an error; a failure directly marks the pipeline as **FAILURE** and normally halts later stages. In short: skipped means “deliberately not run,” failed means “run, and it broke.”

Question 7 — Two Milestones Guarding Build and Deploy

app.py

```
def main():
    print("Deployment executing")

if __name__ == "__main__":
    main()
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                sleep(time: 20, unit: 'SECONDS')
                milestone(1)
            }
        }
        stage('Deploy') {
            steps {
                milestone(2)
                echo 'Deploying application to the target environment...'
            }
        }
    }
}
```

Clicking Build Now twice in quick succession. Say build #1 starts first and enters its 20-second sleep in Build. Before it finishes, build #2 is triggered and runs the same short Build stage. Because milestone(N) only ever lets the *most recently started* build that reaches milestone

N proceed — and automatically cancels any older, still-in-flight build that has not yet passed that same milestone — build #2 reaches milestone(1) while build #1 is still asleep. At that point Jenkins aborts build #1 **at milestone(1)** (it never even finishes its Build stage), while build #2 continues on into Deploy.

What milestone(2) additionally protects. milestone(1) only orders entry into Deploy; it says nothing about what happens *inside* Deploy itself. Placing milestone(2) at the very start of Deploy closes a second race window: if, for some reason, an older build still managed to reach the Deploy stage after a newer build had already started (or was about to start) its own Deploy, milestone(2) guarantees only the newest build is allowed to actually perform the deployment steps — any older one arriving at that checkpoint after a newer one already passed it is aborted right there, before it echoes the deployment message.

Without any milestones. If both overlapping builds were allowed to reach Deploy unguarded, they could run their deployment steps concurrently or out of order. This risks an *older* build's deployment finishing **after** the newer one, effectively overwriting a newer release with older code, as well as two deployments racing against shared resources (e.g. both writing to the same target, restarting the same service, or both trying to acquire the same deployment lock) — producing inconsistent or corrupted deployment state with no guarantee that the most recent, most correct build is the one actually left running in the target environment.